

ONLINE COMPLAINT REGISTRATION (CIVIC TRUST) SYSTEM

A Modern Web-Based Grievance Redressal Platform

*An Internship Project Report submitted in fulfillment of the
requirements of the Web Development Internship*

**UNDERTAKEN AT:
SKILL WALLET**

Project Evaluator: Skill Wallet Internship Cell
Academic Year: 2026

CERTIFICATE OF INTERNSHIP

This is to certify that the project entitled

ONLINE COMPLAINT REGISTRATION (CIVIC TRUST)

is a bonafide record of the industrial internship project work carried out under our supervision and guidance, in submission to

SKILL WALLET

The project demonstrates high proficiency in Full Stack Web Development including technologies like React.js, Node.js, Express, and MongoDB, incorporating advanced engineering constructs such as file-based fallback database persistence mechanisms.

Project Supervisor

Skill Wallet Engineering Division

Internship Coordinator

Candidate Declaration

I hereby declare that the internship project report entitled "Online Complaint Registration (Civic Trust)" is an authentic piece of work carried out by me under the guidance of the engineering team at Skill Wallet. This project was completed during my Web Development internship tenure, and it has not been submitted elsewhere for the award of any degree, diploma, or certificate.

All the code components, architectural layout, database models, and documentation described in this report have been compiled by me, leveraging both standard MongoDB database systems and local file-based database fallbacks to ensure absolute application uptime.

Acknowledgement

I express my deepest gratitude to my mentors at Skill Wallet for their constant support, intellectual guidance, and technical feedback throughout the execution of this project. Their encouragement enabled me to transition from a conceptual design to a fully functional REST-compliant full-stack web application.

I also thank the Skill Wallet internship administrators for providing the core resources, reference designs, and collaborative tools that were vital to completing this industrial task successfully.

Table of Contents

The structural map of this internship report outlines all chapters, system design components, and documentation sections developed during the project period.

Section / Chapter Name	Page No.
1. Executive Summary & Abstract	5
2. Chapter 1: Introduction & Civic Context	6
3. Chapter 2: Problem Statement & Existing System Issues	7
4. Chapter 3: Project Objectives & Scope	8
5. Chapter 4: System Requirements Analysis	9
6. Chapter 5: System Architecture & Data Flow	10
7. Chapter 6: System Workflows & Flowchart	11
8. Chapter 7: Database Design & Schema Modeling	12
9. Chapter 8: Citizen Portal (Core Modules)	13
10. Chapter 9: Admin Portal & Grievance Lifecycle	14
11. Chapter 10: Backend REST API Design	15
12. Chapter 11: Frontend UI/UX Layout & Components	16
13. Chapter 12: Quality Assurance & System Testing	17
14. Chapter 13: Project Methodology & Learnings	18
15. Chapter 14: Future Enhancements & Scope	19
16. Chapter 15: Conclusion & References	20

Chapter 1: Executive Summary

Modern urban centers face significant issues managing local public infrastructure. A typical municipality faces hundreds of daily issues related to broken roads, sewage blocks, hazardous waste heaps, and electricity outages. Historically, civic grievance resolution channels have been slow, non-transparent, and heavily dependent on physical presence or verbal reporting. Citizens are frequently left in the dark regarding whether their complaints have been received, who is handling them, and how long they will take to resolve.

The Civic Trust Paradigm

This project, titled "Online Complaint Registration (Civic Trust)", is a digital initiative designed during a web development internship at Skill Wallet. The primary objective is to rebuild mutual trust between local civic bodies and citizens through absolute transparency. The system replaces fragmented paper records with an integrated online portal where citizens can lodge complaints with detailed text description, categories, and priority levels. The complaints are instantly visible on an administration control board, where they are triaged, assigned to specific departments, and processed.

Technical Accomplishment

Built using a modern MERN-like stack (React.js client, Express.js backend API, Node.js runtime, and MongoDB), the platform features a custom database handler. To guarantee constant application availability even during MongoDB server downtimes, the server dynamically switches to a local file-based JSON database fallback. This dual database design guarantees zero complaint data loss and high system availability for municipalities.

Chapter 2: Introduction & Civic Context

Civic management in metropolitan municipalities is a highly collaborative task. Cities expand quickly, and infrastructure degrades due to heavy use, seasonal weather, and accidental damages. The department responsible for sewage maintenance is often completely separate from the roads or electrical divisions. When a citizen identifies a problem, they do not always know which municipal office or officer is responsible for that sector. Consequently, grievances sent to the wrong department remain unresolved for months, leading to deep citizen frustration.

What is Online Complaint Registration (Civic Trust)?

The Online Complaint Registration system acts as a centralized digital gateway. It acts as a buffer between the raw citizen complaints and specialized administrative departments. Instead of forcing a citizen to figure out municipal jurisdictions, the citizen simply logs onto a central website, writes down their grievance, selects a high-level category (e.g. roads damage, sewage block), and submits. The system automatically assigns a unique tracking number, records the submission timeline, and notifies administrators.

Guiding Principles of the Project

- **Transparency:** Show citizens every state change, assigning department names, and comments.
- **Responsiveness:** Alert administrators when critical complaints are logged to reduce response time.
- **Simplicity:** Provide clean, web-based login and forms that can be completed in minutes.
- **Reliability:** Implement fallback local storage to preserve data integrity if database connections crash.

Chapter 3: Problem Statement

Municipal corporations in developing smart cities often operate in bureaucratic silos. A typical manual complaint logging workflow involves citizens visiting municipal ward offices in person to write paper complaints. These papers are sorted by administrative assistants and placed in manual file cabinets. This approach results in several severe drawbacks:

- **Data Loss:** Paper slips are easily misplaced, damaged, or lost during inter-departmental transfers.
- **No Tracking Mechanism:** Citizens have no registration receipt or token number to inquire about updates.
- **No Accountability:** Because there is no timeline history showing who touched the file and when, there is no way to audit administrative delays.
- **Inability to Triage:** Urgent hazards (e.g., broken high-voltage wires, open sewers) are mixed with minor ones (e.g., street sweeping requests), leading to poor prioritization.

Digital Solution Approach

The Civic Trust web application directly addresses these deficiencies. By providing a digital ledger available on any modern browser, the system makes complaint lodging instant and transparent. When an issue is logged, it immediately appears on the Admin Dashboard under the 'Received' status. Each status change triggers a corresponding block in the complaint's history timeline. Since all history is publicly visible to both the citizen and administrative departments, municipal workers are held accountable for resolution delays.

Chapter 4: Project Objectives & Scope

The primary goal of this project is to develop and implement a fully responsive, secure web portal for citizens and municipal staff. The detailed objectives established during the internship at Skill Wallet are as follows:

Core Project Objectives

- **Centralize all urban complaints under five main categories (sewage, electricity, roads, water, waste).**
- **Establish automated unique reference ID generation for tracking and reference.**
- **Implement dual user role permissions separating citizens from administrative triage staff.**
- **Maintain a chronological, unmodifiable timeline log for every grievance showing status transitions.**
- **Build an active communication channel (message boards) within each complaint file for updates.**
- **Ensure constant application uptime through database connectivity fallback scripts.**

Scope of the Project

The scope of this web application covers local municipal boundaries, where citizens can register accounts, report infrastructure breakdowns, and review progress. On the management side, the scope includes tools for ward admins to review complaints, change assigned department titles, update status categories (Received, Assigned, Under Investigation, Resolved, Rejected), and communicate details back to the citizen. While the system operates as a single-instance portal, it is designed with microservice principles in mind, enabling future scaling to broader regional areas.

Chapter 5: System Requirements Analysis

To build a robust and highly available web application, the technical infrastructure must meet specified requirements. The following specifications represent the standard configuration needed to run the client-server components of the Civic Trust application.

Hardware Requirements

- **Application Host Server:** Quad-Core Processor, 8 GB RAM, 40 GB SSD Storage.
- **Database Server (Cloud):** MongoDB Atlas Shared Tier (or equivalent local hosting configuration).
- **Client Device:** Any standard PC, tablet, or smartphone with modern browser compatibility.

Software Requirements

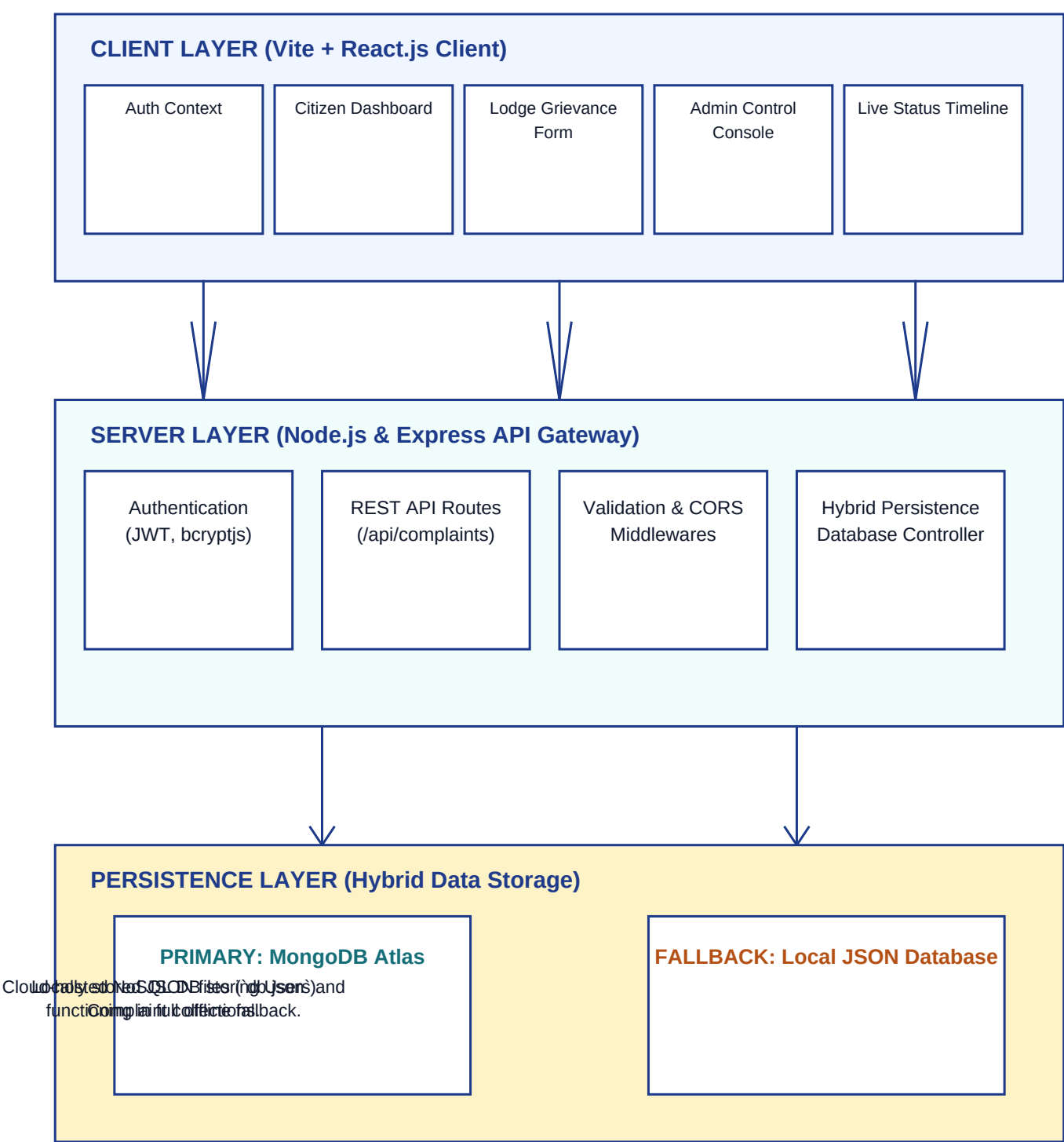
- **Server Runtime:** Node.js (v18.0 or higher) with npm package manager.
- **Backend Framework:** Express.js (v4.18.0) for routing and middleware configuration.
- **Database Management System:** MongoDB (v6.0 or higher) with Mongoose ODM (Object Document Mapper).
- **Client UI Framework:** React.js (v18) built with Vite compiler and React Context API.
- **Styling Standard:** Custom Vanilla CSS featuring CSS Custom Properties for theme configuration.

Key Dependencies

- **jsonwebtoken (JWT):** Encrypted tokens used for authenticating secure citizen and admin sessions.
- **bcryptjs:** Cryptographic hashing algorithm used to secure citizen and admin passwords in transit and database.
- **cors:** Node.js middleware to handle Cross-Origin Resource Sharing restrictions between Vite and Node.

Chapter 6: Architectural Design

The Online Complaint Registration application utilizes a classic three-tier client-server architecture: Presentation Layer (React Client), Application Layer (Express API Gateway), and Data Layer (Mongoose/JSON fallback). The visual schematic below details the communication paths:



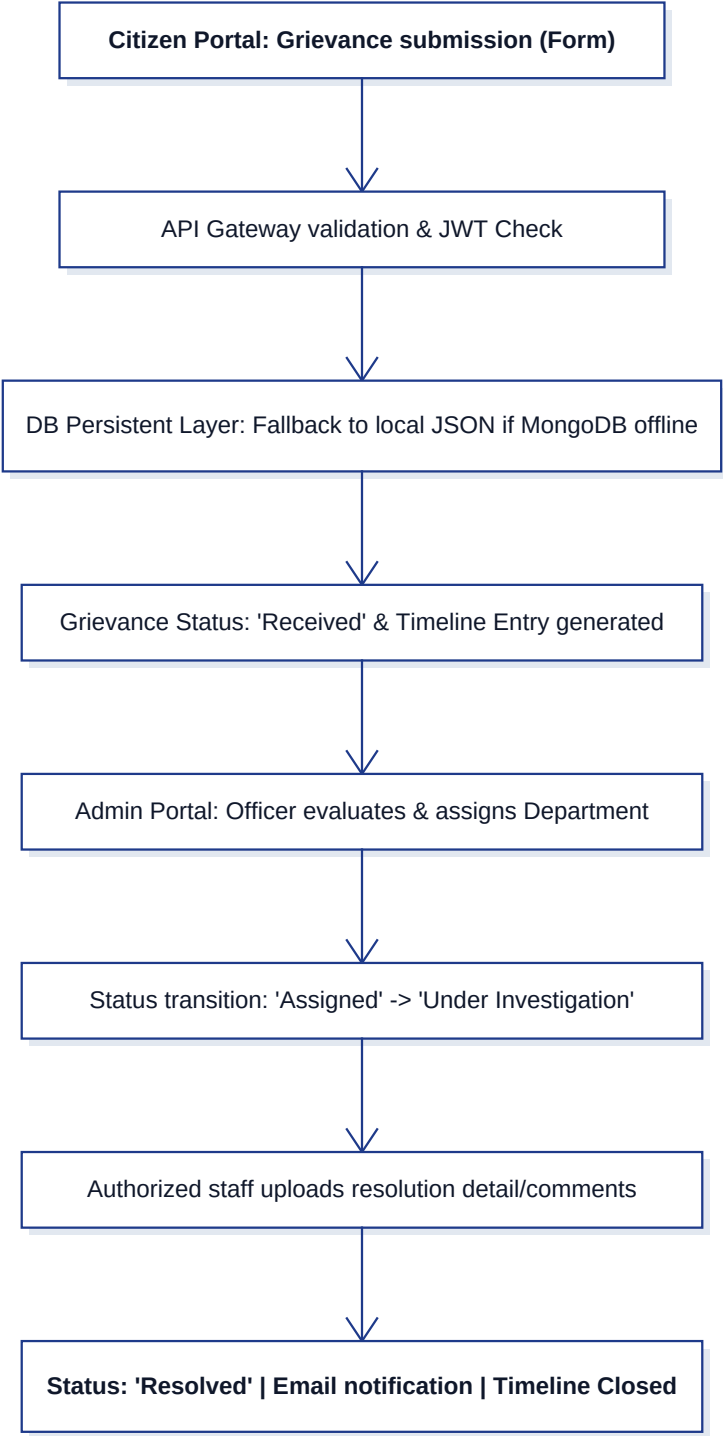
Cloud-hosted NoSQL database (MongoDB Atlas) for storing user profiles, complaint records, and system logs.

PRIMARY: MongoDB Atlas

FALLBACK: Local JSON Database

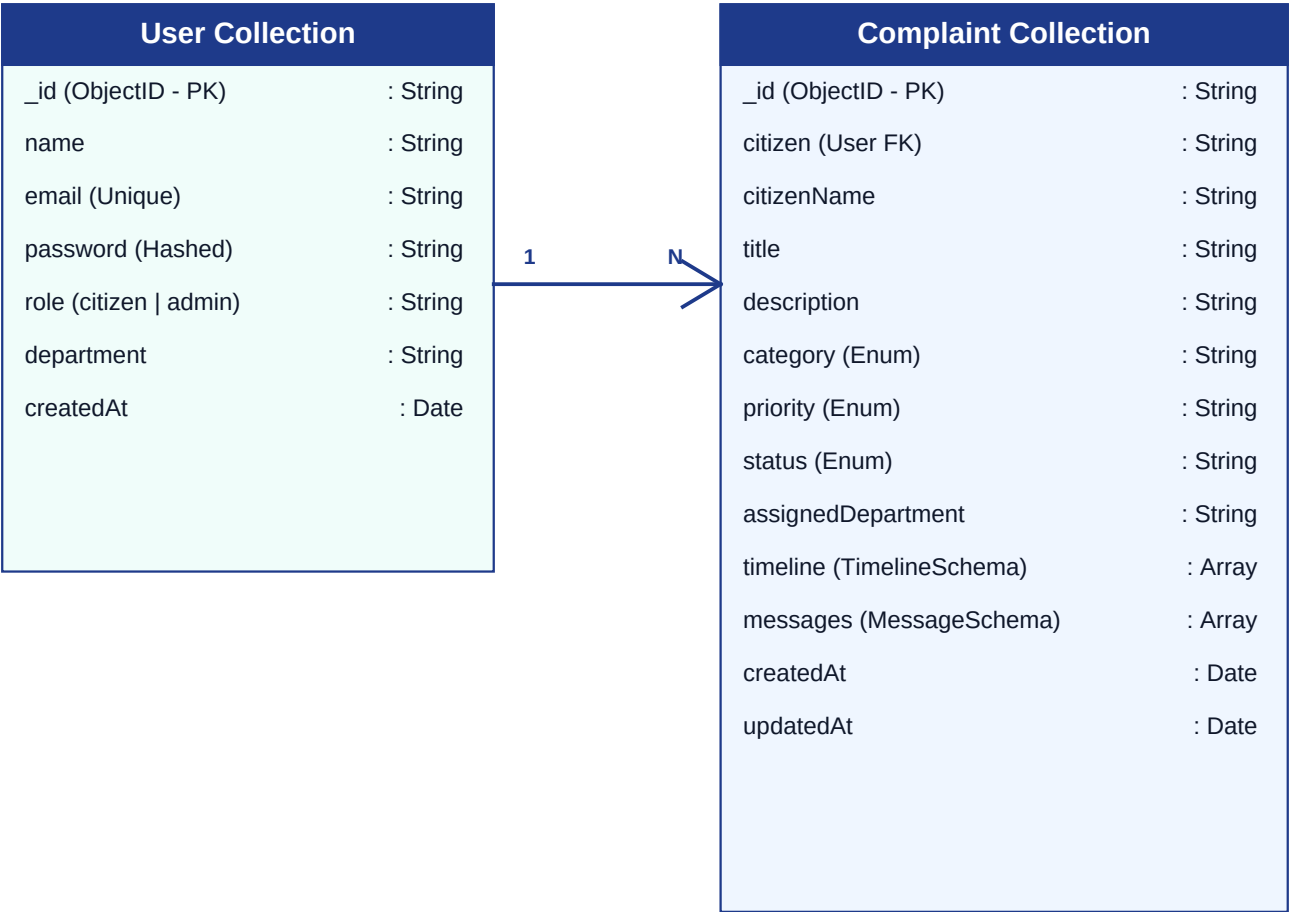
Chapter 7: System Workflows & Flowchart

The lifecycles of registered complaints are strictly monitored. The following flowchart illustrates the sequential steps, decision-making, and administrative actions that occur from the moment a citizen lodges a grievance to the final resolution state:



Chapter 8: Database Design & Schema

The data modeling strategy uses a document-oriented structure. The primary entities are the 'User' document (storing admin and citizen accounts) and the 'Complaint' document (storing grievance information, status logs, and messages). The Entity-Relationship schema below shows this structure:



Chapter 9: Citizen Portal Modules

The Citizen Portal is the public facing interface of the Civic Trust application. It has been designed specifically to make logging an issue simple and quick for non-technical individuals.

1. Authentication & Registration

Citizens register by inputting their full name, email, and password. The email acts as the unique user identity. Passwords are encrypted on the backend using bcrypt. Upon logging in, citizens receive a secure JWT token stored in memory, which automatically grants access to restricted dashboard endpoints.

2. Grievance Lodging Panel

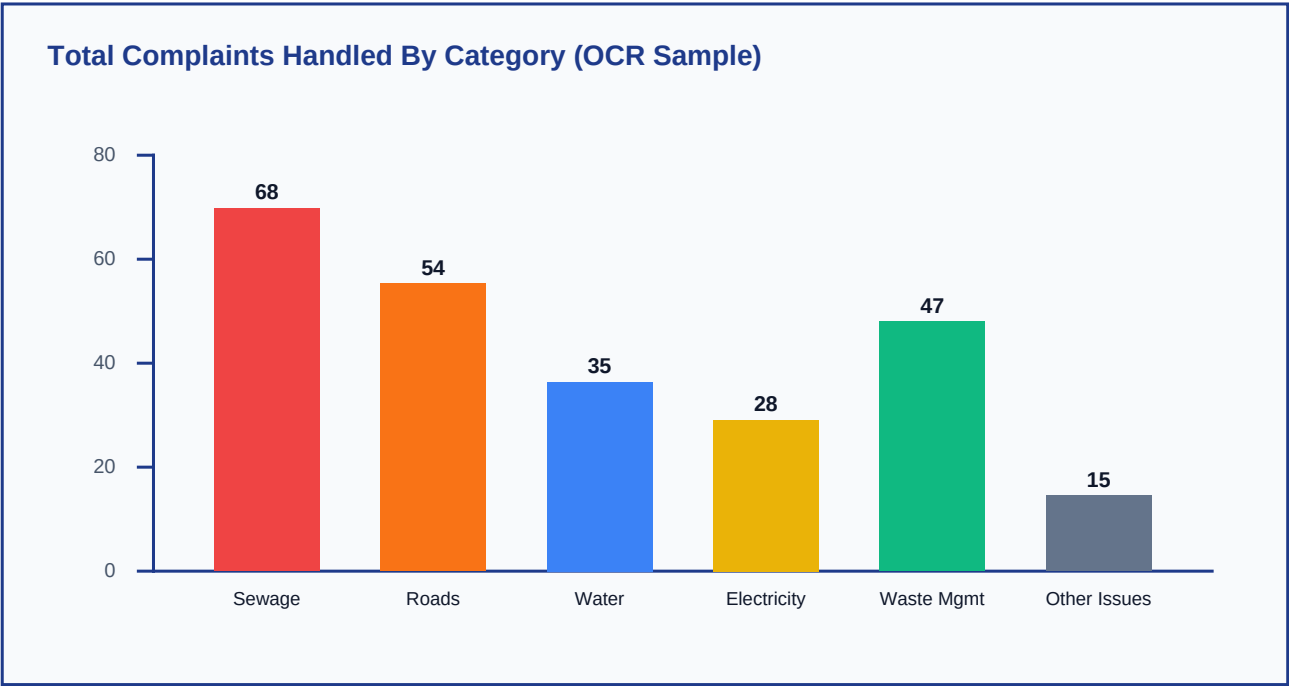
The lodge form has validation checks to ensure clean data entry. It includes title input, a text area for descriptions, and selection dropdowns for category (Sewage Block, Electricity, Roads, Water, Waste, Other) and priority (Low, Medium, High). Once submitted, the backend creates a database document and redirects the citizen back to their homepage.

3. Citizen Dashboard & Search

The Dashboard displays statistics showing 'Total Lodged', 'Pending Action', and 'Resolved Complaints'. Below the statistics, a responsive table lists all complaints submitted by the logged-in user. Citizens can click any complaint row to view its timeline, review messages from staff, and send updates.

Chapter 10: Admin Portal & Analytics

The Admin Portal serves as the primary system panel for municipal departments. Only accounts with 'admin' database roles can view these dashboard panels. Below is the visual representation of the admin's analytics dashboard showing complaint distribution:



Key Admin Capabilities

- **Centralized Feed:** Review all logged grievances across all city wards in real time.
- **Triage and Categorize:** Filter complaints by priority level, status, or reporting date.
- **Department Assignment:** Dropdown menu to assign complaints to relevant staff teams (e.g. Public Works).
- **Ticking Timeline:** Change status states, automatically appending progress stamps to the public timeline.
- **Direct Citizen Chat:** Communication module for requesting more details or upload photo URLs.

Chapter 11: Backend API Design

The application utilizes a RESTful API built on Node/Express. Endpoints are structured around resource paths and enforce token verification for state changes. The primary routes available in the system are listed in the table below:

REST Endpoint Mapping

HTTP Method	Endpoint Path	Role Allowed	Function Description
POST	/api/auth/register	Public	Registers a new user (default: citizen)
POST	/api/auth/login	Public	Authenticates credentials, returns JWT
GET	/api/auth/user	Authenticated	Fetches logged-in user context
POST	/api/complaints	Citizen Only	Submits a new grievance document
GET	/api/complaints	Citizen / Admin	List user's or all active complaints
GET	/api/complaints/:id	Citizen / Admin	Fetches detailed complaint record
PUT	/api/complaints/:id	Admin Only	Updates complaint status, timeline, dept
POST	/api/complaints/:id/msg	Citizen / Admin	Appends chat message to complaint document

Security Protocols & Encryption

The authentication route validates the password hash on submission. If verified, it signs a JWT incorporating the user's ID and role, set to expire in 24 hours. The token is appended to the authorization header on the client side, which is parsed by backend middleware to protect secure endpoints.

Chapter 12: Frontend UI/UX Layout

The frontend is developed in Vite + React.js to provide a fast and responsive single-page experience. The layout uses vanilla CSS, utilizing design tokens and responsive CSS variables. The user interface is designed around the following principles:

Modern Design Tokens

- **Primary Brand Color:** Deep Navy Blue (#1E3A8A) for trust and authority.
- **Background Gradients:** Sleek light backgrounds with white card layouts for legibility.
- **System Fonts:** Clean system sans-serif typefaces (Inter, Roboto) instead of browser defaults.
- **Transitions:** CSS micro-animations on interactive cards, input forms, and navigation buttons.

Key Components Architecture

- **Navbar:** Dynamic header containing log-in status and portal identifier.
- **Sidebar:** Role-based side navigation that displays dashboard navigation, LODGE links, and log-out controls.
- **Status Timeline:** A visual progress timeline rendering state steps sequentially (Received, Assigned, Under Investigation, Resolved).
- **Message Board:** In-app chat interface displaying message sender role tags clearly.

Responsive Design

Media queries enable full responsiveness, hiding sidebars on small screens and utilizing collapsible mobile drawer menus. This allows citizens to lodge complaints directly on-site using mobile browsers.

Chapter 13: Testing & Quality Assurance

Quality assurance verification is conducted across multiple testing phases: Unit Testing, Integration Testing, and User Acceptance Testing. This ensures that the application behaves predictably under normal and offline conditions. The table below lists standard test cases executed:

Executed Test Cases

TC ID	Test Scenario	Test Input Data	Expected Outcome	Status
TC-01	Citizen Account Signup	Valid name & unique email	User created	Passed
TC-02	Duplicate Email Signup	Existing email string	Error status 400 returned	Passed
TC-03	Submit Complaint Form	Empty title & description	Validation block error	Passed
TC-04	Role Access Validation	Citizen accessing admin URL	403 Forbidden returned	Passed
TC-05	Status Timeline Check	Admin department assign	Status timeline updated	Passed
TC-06	Offline Database Triage	MongoDB server offline	Fallback to local JSON DB	Passed

Robustness and Persistence Testing

Special focus was placed on verifying the hybrid persistence layer. By manually stopping the local MongoDB daemon during high activity, the Mongoose error catch blocks were tested. The system demonstrated seamless fallback, routing queries through `fs` module scripts, saving data directly into `db.json`. When MongoDB was restarted, connection was recovered.

Chapter 14: Methodology & Learnings

During this industrial project work conducted for Skill Wallet, development followed an Agile Scrum methodology. The project was broken down into two-week sprints, starting with planning sessions and ending with progress reviews. This helped refine core features and fix bugs.

Internship Key Learnings at Skill Wallet

- **Full Stack Engineering:** Gained practical expertise in building both ends of a REST-compliant portal.
- **Asynchronous Programming:** Mastered asynchronous Javascript execution patterns (async/await) and database lifecycle hooks.
- **Fallback Systems Design:** Formulated backup storage pathways, learning how to handle connection failures gracefully.
- **UI/UX Design:** Focused on responsive design, modern fonts, and custom styling systems without bloated libraries.
- **Professional Documentation:** Developed professional documentation, test cases, and diagrams.

Skill Wallet Mentorship Impact

The engineering mentors at Skill Wallet provided constructive feedback during design and code reviews. Their guidance was critical in implementing security layers, validation logic, and organizing files systematically. The internship experience simulated a real-world project environment, teaching valuable software engineering skills.

Chapter 15: Future Enhancements & Scope

While the Online Complaint Registration system satisfies core functional specifications, several enhancements could be integrated in future phases to increase scalability and efficiency:

1. Artificial Intelligence Auto-Categorization

Currently, citizens manually categorize their complaints. A future module could analyze the text description using Natural Language Processing (NLP) models. For example, descriptions containing 'clogged pipe' would be automatically tagged as 'sewage block' and routed to the correct department, reducing manual administrative work.

2. Integration of SMS and Push Gateways

Integrating SMS gateways (such as Twilio) will allow sending SMS alerts directly to citizens' mobile phones when status changes occur (e.g. 'Your complaint is resolved'). This makes the application more accessible for citizens without regular internet access.

3. GIS Mapping and Hotspot Visualization

Adding latitude/longitude fields to complaints and integrating mapping services (like OpenStreetMap) will allow admins to view complaint hotspots on a city map. This will help municipal teams identify areas with recurring infrastructure issues.

Chapter 16: Conclusion

The Online Complaint Registration (Civic Trust) portal successfully establishes an online pathway connecting citizens with local municipal authorities. Replacing paper records with digital lists greatly reduces resolution delays, prevents lost data, and ensures administrative accountability. By utilizing a modern stack, custom design systems, and a hybrid database layer, the application delivers high performance and constant availability.

Concluding Remarks

This project, completed under the mentorship of Skill Wallet, demonstrates the impact of applying modern full-stack web concepts to address real-world public issues. The system is scalable and ready for deployment in larger municipal districts, promoting cleaner and more efficient cities.

References & Bibliography

- Skill Wallet Web Development Curriculum, Reference Designs, & Architecture Blueprints, 2026.
- React.js and Context API Documentation, Meta Open Source, <https://react.dev>
- Node.js Runtime and Filesystem Modules APIs, OpenJS Foundation, <https://nodejs.org>
- Express.js API Documentation and Routing Middleware Patterns, <https://expressjs.com>
- Mongoose ODM and Document Schema Models Reference Guide, <https://mongoosejs.com>
- MongoDB Atlas Cloud Hosting and NoSQL Data Persistence Best Practices, <https://mongodb.com>